

Parallel Modular Exponentiation

Ken Duculan

Mark Cuizon

CS 171

1. RSA and the Core Operation
2. The Three Approaches
3. Approach C: Parallel Precomputation
4. Metrics

RSA and the Core Operation

The RSA Algorithm

RSA secures HTTPS, banking, and messaging. It has three steps:

1. Key Generation

1. Pick two large primes p, q
2. Compute modulus $n = p \times q$
3. Derive public exponent e and private exponent d
4. Publish (e, n) ; keep (d, n) secret

2. Encryption

1. Represent message as integer m
2. Compute with **public key**:
$$c = m^e \bmod n$$
3. Transmit ciphertext c

3. Decryption

1. Apply **private key**:
$$m = c^d \bmod n$$
2. Original message recovered

Both steps use the **same core op.**

The Bottleneck

A 2048-bit key requires $\approx 3,000$ multiplications per message. At server scale (millions of requests/sec), this is a significant cost.

How Exponentiation Works — Naive vs. Binary

Naive: multiply m by itself e times. For a 2048-bit exponent that is 2^{2048} steps — infeasible.

Binary exponentiation: write e in binary, then only *square* and *multiply when the bit is 1*.

Example: m^{13} , since $13 = 1101_2$

Bit	Action	Value
1	Start, bit=1 → multiply	m
1	Square, bit=1 → multiply	m^3
0	Square only	m^6
1	Square, bit=1 → multiply	m^{13}

4 steps instead of 13.

Why it is hard to parallelize

Each step uses the result of the **previous step**. Step 3 cannot start until step 2 finishes. This chain forces the algorithm to run **one step at a time** no matter how many cores are available.

Our Goal

Restructure the computation so steps become **independent** and can all run simultaneously on the GPU.

The Three Approaches

Three Ways to Compute $m^e \bmod n$

All three approaches solve the **same problem** — modular exponentiation — but differ in *where* and *how* parallelism is applied.

	(A) Sequential	(B) GPU Batching	(C) Bit-Parallel
Parallelism	None	Across messages	Within one message
Threads used	1 CPU thread	One thread <i>per message</i>	One thread <i>per bit</i>
Dependency chain	Fully sequential	None across threads	Broken in Phase 1
Best case	Single message, simple	Many messages at once	Few messages, large key
Bottleneck	Every step	Memory / scheduling	Final reduction

Key Insight

(B) and (C) target **different axes of parallelism**. Batching scales with the *number of inputs*; bit-parallel scales with the *size of the exponent*. They are complementary, not competing.

Approach B — GPU Batching

Idea: keep binary exponentiation unchanged. Instead of parallelizing *inside* one operation, run **many independent operations at once**.

Each GPU thread owns one full RSA operation:

- Thread 0 $\rightarrow$ encrypt m_0
- Thread 1 $\rightarrow$ encrypt m_1
- $\vdots$
- Thread k $\rightarrow$ encrypt m_k

Zero inter-thread communication

Near-linear speedup (up to GPU saturation)

When It Wins

GPU batching excels when the workload is **many messages** processed together (e.g. a TLS server handling thousands of handshakes per second). Utilization is very high because every thread does useful work.

When It Falls Short

If only **one or a few messages** must be encrypted, most threads sit idle. The GPU is under-utilised and offers little advantage over the CPU.

Approach C: Parallel Precomputation

The Key Insight — Split Work by Bit Position

Observation

m^{11} can be rewritten as $m^8 \times m^2 \times m^1$ — one term per set bit in $11 = 1011_2$. Each term depends **only on m and n** , never on the other terms.

Assign one GPU thread per bit. All threads compute their term **simultaneously**:

Thread	Term	Bit in $11 = 1011_2$	Status
0	m^1	bit 0 = 1	Contribute to answer
1	m^2	bit 1 = 1	Contribute to answer
2	m^4	bit 2 = 0	Discard
3	m^8	bit 3 = 1	Contribute to answer

Why This Breaks the Dependency

No thread waits for another. All four run at once. Multiply the kept results to get the final answer.

Two-Phase Algorithm

Phase 1 — Parallel (GPU)

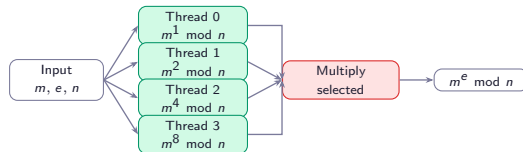
One CUDA thread per bit of e . Each independently computes its power of $m \pmod n$ by local squaring.

No inter-thread communication (Completely Independent)

Phase 2 — Combine

Collect results where the bit was 1. Multiply them in a **tree pattern**

Sequential step



Worked Example: $m = 3$, $e = 11$, $n = 17$

Step 0. $e = 11 = 1011_2 \Rightarrow$ bits set at positions 0, 1, and 3.

Phase 1 — All four threads run simultaneously (mod 17):

Thread	Bit	Computation	mod 17	
0	1	$3^1 = 3$	3	keep
1	1	$3^2 = 9$	9	keep
2	0	$3^4 = 81$	$81 - 4 \cdot 17 = 13$	discard
3	1	$3^8 = 13^2 = 169$	$169 - 9 \cdot 17 = 16$	keep

Phase 2 — Multiply the kept results:

$$3 \times 9 \times 16 = 432 \quad 432 = 25 \times 17 + 7 \Rightarrow 3^{11} \bmod 17 = 7$$

Verification

$$3^{11} = 177,147 \quad \text{and} \quad 177,147 = 10,420 \times 17 + 7 \quad \checkmark$$

Metrics

Metric	
Speedup	Times faster than the sequential baseline.
Work Efficiency	Whether added threads do useful work or sit idle.
Amdahl's Law	Hard ceiling on speedup from the sequential combine step.
Thread Divergence	How often GPU threads stall from different code paths.
CGMA	Whether the GPU computes more than it fetches from memory.